

CHAPTER III

HDFS Architecture & Data Storage

1. PRESENTATION OF HDFS
2. HDFS ARCHITECTURE
3. ROBUSTNESS
4. DATA ORGANIZATION

HDFS Architecture & Data Storage

Objectives

Section 01

- 1 Understand the Fundamentals of HDFS
Explain the role of HDFS in the Hadoop ecosystem.
Describe the need for a distributed file system.
- 2 Explore HDFS Architecture
Identify the key components of HDFS and explain the responsibilities of each component.
Understand how metadata management and namespace operations work in HDFS.
- 3 Learn HDFS Data Storage Mechanism
Describe how HDFS stores large files as blocks across multiple nodes.
Understand block size configuration and its impact on performance.

Fouad DAHAK
ESIA, 2024/2025

CHAPTER III : HDFS ARCHITECTURE & DATA STORAGE

Section N° 1

HDFS PRESENTATION

Fouad DAHAK, ENSIA, 2024/2025

HDFS Architecture & Data Storage

HDFS Overview

Section 01



HDFS
Distributed file system that provides high-throughput access to application data.

Suitable for the distributed storage and processing	Files and folders are organized in a tree (like Unix) and stored on a large number of machines in a way to make the exact position of a file invisible.
Built on the architecture of Google File System (GFS)	
Shares a similar architecture to many other common distributed storage engines such as Amazon S3 and Microsoft Azure	
A stand-alone storage engine and can be used in isolation of the query processing engine	Access is transparent, regardless of the machines that contain the files.
Designed to run on a cluster of commodity hardware	Files are copied in multiple copies for reliability and to allow multiple simultaneous accesses
Pattern emphasis: Write Once, Read Many	
Very Large Files: One file could span the entire HDFS	
HDFS allows to see all the folders and files of these thousands of machines as a single tree, containing Po of data, as if they were on the local hard drive.	

Fouad DAHAK
ESIA, 2024/2025

Section 01

HDFS Architecture & Data Storage

HDFS is not Good for



- Many, many small files**: Scalability issue for the namenode (more in a moment)
- Low Latency Access**: It's all about Throughput not about minimizing service time (or latency to first data read)
- Multiple writers**: updates at offsets within a file. One writer Create/rename/move/delete No updates!
- Not a substitute for a relational database**: Data stored in files, not indexed. To find something, must read all the data (ultimately, by a MapReduce job/tasks)

Fouad DAHAK ENSIA, 2024/2025

Section 01

HDFS Architecture & Data Storage

HDFS Architectural Concepts & Principles

Assumptions and Goals	Principles
Hardware failure is the norm	Simple coherency model
Fault-tolerance in the face of massive parallelism (many devices, high probability of HW failure)	HDFS applications need a write-once-read-many access (WORM) model for files. A file once created, written, and closed need not be changed except for appends and truncates.
Monitoring, detection, fast automated recovery	
Batch processing (batch throughput)	Master-slave: Decouples metadata from data operations
Hence support for very large files; large block size; single writer, create or append (no update) and a high read ratio	One NameNode manages the filesystem and client access (All metadata - namespace tree & map of file blocks to DataNodes) DataNodes manage locally attached storage.
Large datasets	Replication provides reliability and high availability
HDFS is tuned to support large files. It should provide high aggregate data bandwidth and scale to hundreds of nodes in a single cluster. It should support tens of millions of files in a single instance.	Network traffic minimized
	Not layered on or dependent on any other filesystem

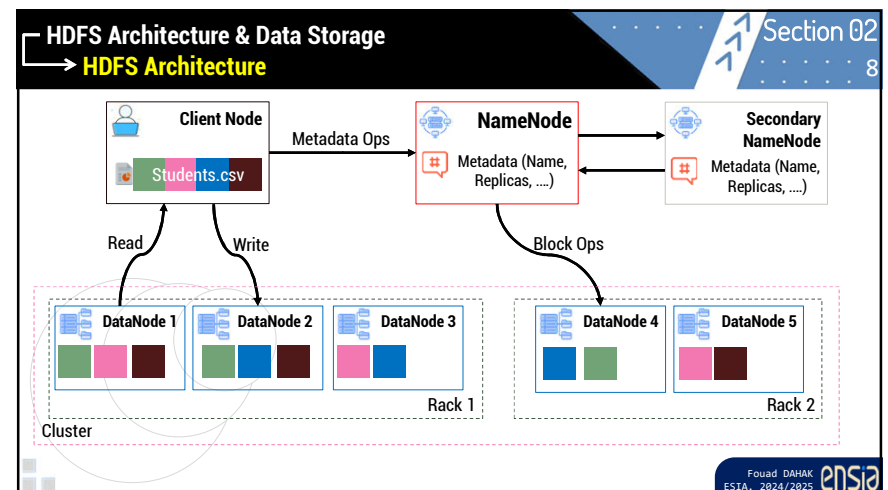
Fouad DAHAK ENSIA, 2024/2025

CHAPTER III : HDFS ARCHITECTURE & DATA STORAGE

Section N° 2

HDFS ARCHITECTURE

Fouad DAHAK, ENSIA, 2024/2025



HDFS Architecture

Namenode and Datanodes



Master/slave architecture

HDFS cluster consists of a single Namenode, a master server that manages the file system namespace and regulates access to files by clients.

There are a number of DataNodes usually one per node in a cluster.

The DataNodes manage storage attached to the nodes that they run on.

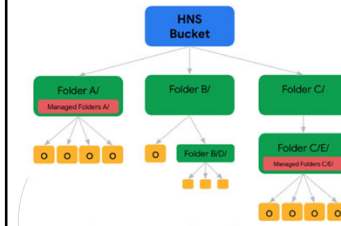
HDFS exposes a file system namespace and allows user data to be stored in files.

A file is split into one or more blocks and set of blocks are stored in DataNodes.

DataNodes: serves read, write requests, performs block creation, deletion, and replication upon instruction from Namenode.

HDFS Architecture

File system Namespace



Hierarchical file system with directories and files

Create, remove, move, rename etc.

Namenode maintains the file system

Any meta information changes to the file system recorded by the Namenode.

An application can specify the number of replicas of the file needed: replication factor of the file. This information is stored in the Namenode.

HDFS Architecture

Data Replication

HDFS is designed to store very large files across machines in a large cluster.

Each file is a sequence of blocks.

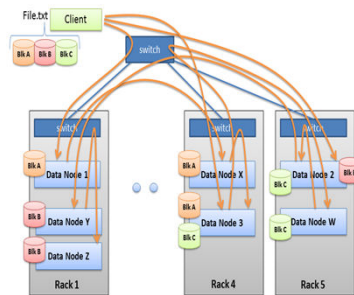
All blocks in the file except the last are of the same size.

Blocks are replicated for fault tolerance.

Block size and replicas are configurable per file.

The Namenode receives a Heartbeat and a BlockReport from each DataNode in the cluster.

BlockReport contains all the blocks on a Datanode.



HDFS Architecture

Replica Placement

The placement of the replicas is critical to HDFS reliability and performance.

Optimizing replica placement distinguishes HDFS from other distributed file systems.

Rack-aware replica placement: improve reliability, availability and network bandwidth utilization

Many racks, communication between racks are through switches.

Network bandwidth between machines on the same rack is greater than those in different racks.

Namenode determines the rack id for each DataNode.

Replicas are typically placed on unique racks :- Simple but non optimal - Writes are expensive - Replication factor is 3

Replicas are placed: one on a node in a local rack, one on a different node in the local rack and one on a node in a different rack.

HDFS Architecture

Replica Selection

Replica selection for READ operation: HDFS tries to minimize the bandwidth consumption and latency.

If there is a replica on the Reader node then that is preferred.

HDFS cluster may span multiple data centers: replica in the local data center is preferred over the remote one.

HDFS Architecture

Safemode Startup

On startup Namenode enters Safemode.

Replication of data blocks do not occur in Safemode.

Each DataNode checks in with Heartbeat and BlockReport.

Namenode verifies that each block has acceptable number of replicas

After a configurable percentage of safely replicated blocks check in with the Namenode, Namenode exits Safemode.

It then makes the list of blocks that need to be replicated.

Namenode then proceeds to replicate these blocks to other Datanodes.

HDFS Architecture

Filesystem Metadata

The HDFS namespace is stored by Namenode.

Namenode uses a transaction log called the **EditLog** to record every change that occurs to the filesystem meta data.

Entire filesystem namespace including mapping of blocks to files and file system properties is stored in a file **Fslmage**.

The footer shows 'Fouad DAHAK ENSIA, 2024/2025' and the 'ensia' logo.

HDFS Architecture

Namenode

Keeps image of entire file system namespace and file Blockmap in memory.

4GB of local RAM is sufficient to support the above data structures that represent the huge number of files and directories.

When the Namenode starts up it gets the Fslmage and Editlog from its local file system, update Fslmage with EditLog information and then stores a copy of the Fslmage on the filesystem as a checkpoint.

Periodic checkpointing is done. So that the system can recover back to the last checkpointed state in case of a crash.

The footer shows 'Fouad DAHAK ENSIA, 2024/2025' and the 'ensia' logo.

Section 02

Datanode has no knowledge about HDFS filesystem

It stores each block of HDFS data in a separate file.

Datanode does not create all files in the same directory.

It uses heuristics to determine optimal number of files per directory and creates directories appropriately.

When the filesystem starts up it generates a list of all HDFS blocks and send this report to Namenode: Blockreport.

Section N° 3

HDFS ROBUSTNESS

Section 03

HDFS

Primary objective of HDFS is to store data reliably in the presence of failures.

Namenode failure

Datanode failure

Network partition

Section 03

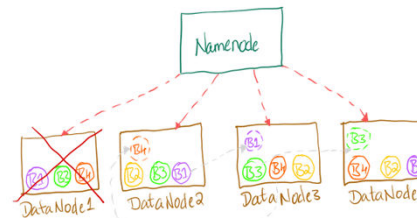
A network partition can cause a subset of Datanodes to lose connectivity with the Namenode.

Namenode detects this condition by the absence of a Heartbeat message.

Namenode marks Datanodes without Heartbeat and does not send any IO requests to them.

Any data registered to the failed Datanode is not available to the HDFS.

Also the death of a Datanode may cause replication factor of some of the blocks to fall below their specified value.



DATANODE FAILURE

Robustness
→ **Re-replication**

Section 03

The necessity for re-replication may arise due to:

- A Datanode may become unavailable,
- A replica may become corrupted,
- A hard disk on a Datanode may fail, or
- The replication factor on the block may be increased.

Fouad DAHAK
ESTIA, 2024/2025



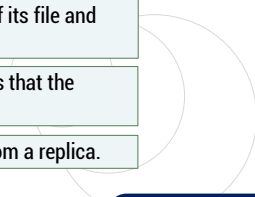
Robustness
→ **Data Integrity**

Section 03

Consider a situation: a block of data fetched from Datanode arrives corrupted.

- This corruption may occur because of faults in a storage device, network faults, or buggy software.
- A HDFS client creates the checksum of every block of its file and stores it in hidden files in the HDFS namespace.
- When a client retrieves the contents of file, it verifies that the corresponding checksums match.
- If does not match, the client can retrieve the block from a replica.

Fouad DAHAK
ESTIA, 2024/2025



Robustness
→ **Metadata Disk Failure**

Section 03

FsImage and EditLog are central data structures of HDFS.

A corruption of these files can cause a HDFS instance to be non-functional.

- For this reason, a Namenode can be configured to maintain multiple copies of the FsImage and EditLog.
- Multiple copies of the FsImage and EditLog files are updated synchronously.

The Namenode could be single point failure: automatic failover is NOT supported!

Fouad DAHAK
ESTIA, 2024/2025



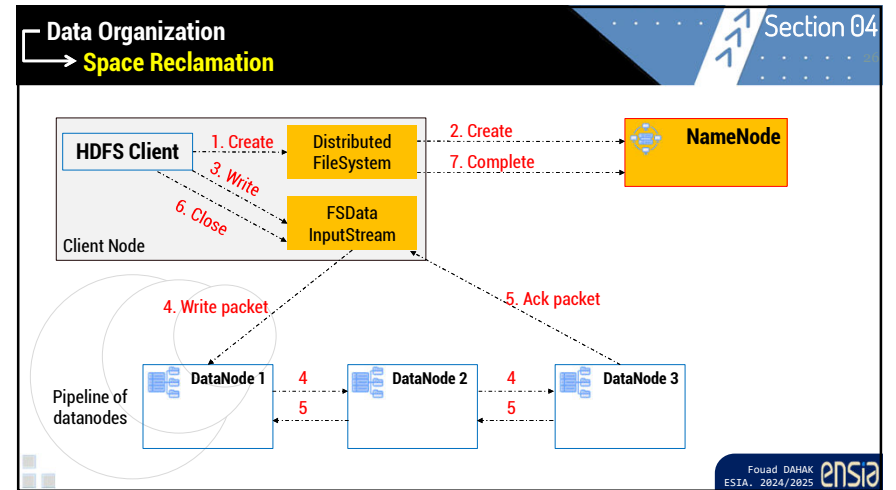
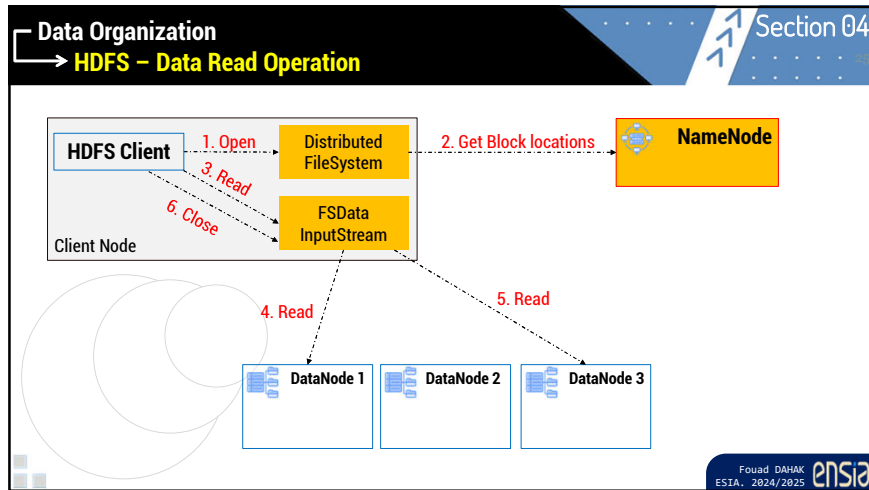
CHAPTER III : HDFS ARCHITECTURE & DATA STORAGE

**Section
N° 3**

**DATA
ORGANIZATION**

Fouad DAHAK, ENSIA, 2024/2025





Chapter II : Big Data Ecosystem & Technologies
→ **Summary and Q&A**

27

Thank You

Q & A

Fouad DAHAK
ESIA, 2024/2025
ensia